



合肥工业大学
HEFEI UNIVERSITY OF TECHNOLOGY

书店图书管理系统实现

第 3 小组

组长：贺鑫帅

组员：待定

数据库系统课程设计答辩

合肥工业大学

2026.6.18

汇报结构

1. 项目目标与实现范围
2. 技术栈与运行环境
3. 角色权限与业务流程设计
4. 数据库对象与安全机制
5. 系统运行与界面展示
6. 不足总结与优化方向



项目目标与实现范围

项目目标

实现图书类别、出版社、图书、仓库信息管理，并完成进货、入库、销售、出库等业务。

系统扩展

增加多角色入口、库存预警、畅销排行、财务审核和区间统计查询。

实现方式

Flask 负责 Web 服务，Jinja2 渲染页面，psycopg2 连接华为云 GaussDB。



核心：让采购、库存、销售、财务统计围绕同一套数据库状态闭环运转。

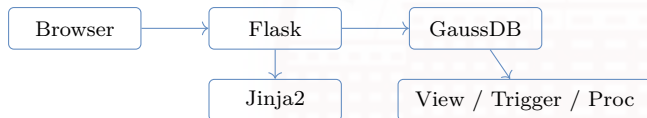
技术栈与系统架构

B/S 架构

- ▶ 浏览器访问 Flask Web 服务
- ▶ Jinja2 模板渲染业务页面
- ▶ Bootstrap 辅助响应式布局
- ▶ 角色主页按业务入口组织功能

后端与数据库

- ▶ Python 3.12 本地虚拟环境运行
- ▶ Flask-Login 维护登录态
- ▶ psycopg2 连接 GaussDB
- ▶ 触发器、视图、函数和存储过程承担数据层逻辑



角色权限设计

五类角色

- ▶ 系统管理员：全表管理、用户权限、统计看板
- ▶ 采购员：出版社维护、采购订单、补货参考
- ▶ 仓库管理员：库存更新、图书查询、库存预警
- ▶ 销售员：销售开单、库存查询、畅销排行
- ▶ 财务人员：明细审核、采购价监督、统计报表

设计原则

- ▶ 按角色暴露页面入口，降低误操作概率
- ▶ 采购、销售、库存、财务动作相互衔接
- ▶ 关键库存约束下沉到数据库层维护

页面路由

登录后根据用户角色跳转到对应 dashboard，并在业务路由中进行权限检查。

权限矩阵与角色边界

角色入口

角色	主要业务空间
系统管理员	全表管理、用户权限、统计看板
采购员	出版社维护、采购订单、补货参考
仓库管理员	库存维护、图书查询、库存预警
销售员	销售开单、库存查询、畅销排行
财务人员	订单审核、采购价监督、区间统计

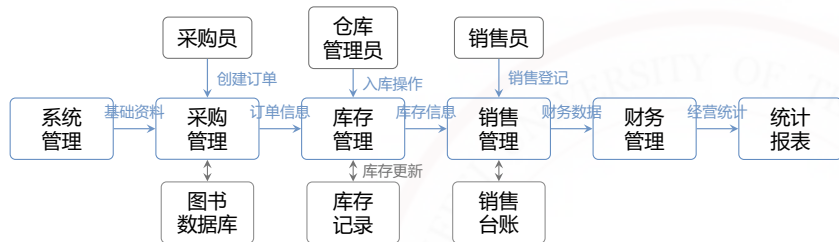
隔离方式

- ▶ 登录后依据 `current_user.role` 路由到对应主页
- ▶ 每个业务路由二次校验角色，越权返回 403
- ▶ 数据库层保留触发器和约束，避免绕过页面造成库存错误

安全收益

页面入口与数据库约束共同限制误操作，降低采购、销售、库存和财务流程互相污染的风险。

业务数据流



采购入库

采购明细写入后，触发库存数量自动增加。

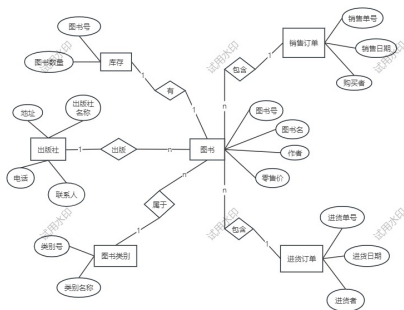
销售出库

销售前检查库存存在性与数量是否充足。

统计分析

视图与函数复用业务数据，支撑预警和经营统计。

逻辑结构与 E-R 模型



核心实体

图书、图书类别、出版社、库存、进货订单、销售订单及对应明细。

规范化

关系模式经规范化分析，均达到 BCNF。

一单多书

订单表保存业务主信息，明细表保存多本图书的数量与价格。

关系模式设计

基础资料

- ▶ 出版社：编号、名称、地址、电话、联系人
- ▶ 图书类别：类别编号、类别名称
- ▶ 图书：图书号、书名、作者、单价、出版社、类别
- ▶ 库存：图书号、库存数量、最后更新时间

规范化结果

基础资料与业务订单拆分存储，避免图书、出版社、类别信息在订单明细中重复维护，核心关系达到 BCNF。

采购与销售

- ▶ 进货订单：进货单号、供货方、进货日期、备注
- ▶ 进货明细：明细号、进货单号、图书号、数量、进货价格
- ▶ 销售订单：销售单号、购买者、销售日期、备注
- ▶ 销售明细：明细号、销售单号、图书号、数量、销售价格

一单多书

订单表保存业务主信息，明细表保存多本图书的数量与价格，便于批量采购、批量销售和后续统计。

三张核心表设计

1. book 图书档案

- ▶ 主键: book_id
- ▶ 外键: 出版社、图书类别; 字段含书名、作者、零售价
- ▶ 作用: 采购、库存、销售共同引用的主数据

2. inventory 库存记录

- ▶ 主键/外键: book_id
- ▶ 字段: 库存数量、更新时间; 库存数量非负
- ▶ 作用: 反映每本图书当前库存, 由触发器维护

3. purchase_detail 采购明细

- ▶ 主键: item_id
- ▶ 外键: 采购订单、图书档案; 字段含数量、采购价
- ▶ 作用: 支持一单多书并触发入库



规范化与索引设计

关系模式

- ▶ 图书、类别、出版社、库存分表维护基础资料
- ▶ 采购订单/明细、销售订单/明细支持一单多书
- ▶ 经规范化分析，核心关系达到BCNF

索引策略

- ▶ 主键自动建立索引，保障实体定位效率
- ▶ 外键字段建立索引：
book.publisher_id、
book.category_id
- ▶ 高频筛选字段建立索引：书名检索、低库存筛选

数据库设计重点不是把表列出来，而是让订单明细、库存变动和统计查询能稳定衔接。

序列与编号生成

序列用途

- ▶ 序列类似 MySQL 的自增字段，但不依赖单张表
- ▶ 可设置初始值、步长、最小值、最大值和循环策略
- ▶ 订单和明细编号由数据库统一生成，减少并发冲突

系统序列

- ▶ `order_seq`: 采购订单和销售订单编号
- ▶ 采购明细编号:
`purchase_detail_item_id_seq`
- ▶ 销售明细编号:
`sales_detail_item_id_seq`

订单编号由数据库层生成，业务层只负责提交订单内容。

连接、安全与数据访问

数据访问层

db.py 封装查询和更新，使用 with 管理连接与游标，并保留异常反馈。

数据访问层：db.py

- 保证数据库连接(connect)与游标(cursor)的**正确打开与关闭**（通过with）
- **异常捕获与处理**（通过try-except）

```
def execute_query(sql, params=None, fetchone=False):
    """执行查询语句，返回全部结果或单条结果"""
    try:
        with psycopg2.connect(**config) as conn:
            with conn.cursor() as cursor:
                cursor.execute(sql, params)
                if fetchone:
                    return cursor.fetchone()
```

```
def execute_update(sql, params=None):
    """执行插入、更新、删除等写操作，返回受影响行数"""
    try:
        with psycopg2.connect(**config) as conn:
            with conn.cursor() as cursor:
                cursor.execute(sql, params)
                affected = cursor.rowcount
```

登录安全

用户密码使用 Werkzeug 哈希存储，登录时进行密文比对。

- 修改**通信协议加密方式**

- GaussDB采用SHA256加密方式 => 无法使用psycopg2 直接连接

```
psycopg2.OperationalError: connection to server at "xxx.xxx.xxx.xxx", port 8000 failed: none of the
server's SASL authentication mechanisms are supported
```

（服务器不支持任何SASL认证机制）

- 修改通信协议加密方式，采用**SHA256**和**MD5**两种方式分别对密码加密



```
def login(username, password):
    """登录验证"""
    # 生成加密后的密码
    encrypted_password = hashlib.sha256(password.encode()).hexdigest()
    # 查询数据库
    sql = "SELECT * FROM user WHERE username = %s AND password = %s"
    params = (username, encrypted_password)
    cursor.execute(sql, params)
    user = cursor.fetchone()
    if user:
        return user
    return None
```

权限隔离

Flask-Login 维护会话；
业务页面根据 current_user.role 做访问控制。

GAUSSDB 连接问题处理

连接方式

- ▶ GaussDB 作为云端主数据库，通过公网 IP 连接
- ▶ Flask 后端使用 psycopg2 执行查询和更新
- ▶ db.py 统一封装连接、游标、提交和异常反馈

协议与认证

- ▶ 默认 SHA256 认证方式会导致 psycopg2 直接连接失败
- ▶ 错误表现为服务器不支持 SASL 认证机制
- ▶ 修改通信协议加密方式，启用 SHA256 与 MD5 兼容

这部分是原 PPT 中数据库连接调试的关键经验，保留为答辩问到部署问题时的依据。

触发器、序列与存储过程

1. 编写触发器函数，定义触发器逻辑

```

CREATE OR REPLACE FUNCTION public.update_inventory_on_purchase()
RETURNS TRIGGER
LANGUAGE PLPGSQL
NOT FENCED NOT SHIPPABLE
AS $function$
DECLARE
    -- 触发前库存是否足够
    current_stock INT;
BEGIN
    IF NOT EXISTS (SELECT 1 FROM inventory WHERE book_id = NEW.book_id) THEN
        -- 触发前库存不足，插入库存记录
        INSERT INTO inventory (book_id, stock_quantity, last_updated)
        VALUES (NEW.book_id, NEW.quantity, NOW());
    ELSE
        -- 触发前库存足够，则更新库存记录
        UPDATE inventory
        SET stock_quantity = stock_quantity + NEW.quantity,
            last_updated = NOW()
        WHERE book_id = NEW.book_id;
    END IF;
    RETURN NEW;
END $function$;

```

TRIGGER

采购后更新库存；销售前校验库存。

```

CREATE OR REPLACE PROCEDURE public.generate_sales_order(
    customer VARCHAR,
    note TEXT,
    book_id VARCHAR,
    quantities INT[],
    prices NUMERIC[],
    sales_date TIMESTAMP
)
AS
DECLARE
    order_id VARCHAR := 'SO-' || nextval('order_seq');
    detail_id INT;
BEGIN
    INSERT INTO sales_order (order_id, customer, sale_date, notes)
    VALUES (order_id, customer, sales_date, note);

```

PROCEDURE + SEQUENCE

生成订单并调用 order_seq。

```

CREATE OR REPLACE FUNCTION public.update_inventory_on_sale()
RETURNS TRIGGER
LANGUAGE PLPGSQL
NOT FENCED NOT SHIPPABLE
AS $function$
DECLARE
    current_stock INT;
BEGIN
    -- 触发前库存是否足够
    SELECT stock_quantity INTO current_stock
    FROM inventory
    WHERE book_id = NEW.book_id;
    FOR UPDATE; -- 更新库存，防止并发冲突
    -- 触发前库存是否足够
    IF current_stock IS NULL THEN
        RAISE EXCEPTION '图书ID % 不存在于库存表中', NEW.book_id;
    ELSEIF current_stock < NEW.quantity THEN
        RAISE EXCEPTION '库存失败：图书 % 库存不足（当前：%, 需求：%)',
            NEW.book_id, current_stock, NEW.quantity;
    END IF;
    -- 更新库存（减少出库量）
    UPDATE inventory
    SET stock_quantity = stock_quantity - NEW.quantity,
        last_updated = NOW();

```

BEFORE SALE

销售前校验库存并扣减。

触发器实现细节

创建步骤

- ▶ 编写触发器函数，定义库存更新或校验逻辑
- ▶ 创建触发器，并绑定到采购明细或销售明细表
- ▶ 触发器逻辑与触发器定义分离，便于复用和维护

系统触发器

- ▶ `trg_after_purchase`: 新增进货记录后自动更新 `inventory`
- ▶ `trg_before_sale`: 销售前校验库存并扣减库存
- ▶ 修改库存逻辑时只需调整函数，无需重新定义触发器入口

销售异常处理

异常场景

- ▶ 当销售的图书不存在于库存表中，触发器抛出异常
- ▶ 当库存数量小于出库数量，触发器拒绝本次销售
- ▶ 查询库存时使用 FOR UPDATE 加锁，减少并发冲突

实现价值

- ▶ 异常处理下沉到数据库层，避免绕过页面造成库存错误
- ▶ PL/pgSQL 过程化逻辑更接近实际业务规则
- ▶ 前端只负责展示结果，库存一致性由数据库保证

视图、函数与统计

共享视图

- ▶ v_best sellers: 按月展示畅销图书
- ▶ v_low_quantity_books: 筛选低库存图书
- ▶ v_num_book_cat: 按类别汇总库存
- ▶ v_publisher_supply: 查看出版社供货情况

函数与过程

- ▶ get_book_stats_by_period: 区间统计查询
- ▶ generate_purchase_order: 生成采购订单
- ▶ generate_sales_order: 生成销售订单

系统不只维护数据，也能为采购、销售和财务判断提供依据。

函数与存储过程对比

函数 FUNCTION

- ▶ `get_book_stats_by_period` 接收开始时间和结束时间
- ▶ 返回指定时间段内各图书进货、销售和库存变化
- ▶ 更适合查询、计算和直接返回结果集

过程 PROCEDURE

- ▶ `generate_purchase_order` 生成采购订单和明细
- ▶ `generate_sales_order` 生成销售订单和明细
- ▶ 更适合执行一组复杂数据操作，内部使用 FOR-LOOP 和序列

函数偏向“查询并返回结果”，过程偏向“执行一组业务写入”。

运行方式与演示账号

本地启动

```
cd
D:\BookstoreManagementSystem
.\.venv\Scripts\python.exe -m
flask --app app:app run
--host 127.0.0.1 --port 5000
```

访问入口

浏览器打开

<http://127.0.0.1:5000/login>

验证: /login 返回 200; admin1 跳转 /admin; GaussDB select 1 成功。

演示账号

角色	账号	密码
系统管理员	admin1	admin123
采购员	proc1	proc123
仓库管理员	ware1	ware123
销售员	sale1	sale123
财务人员	fin1	fin123

登录入口与多角色主页



新版界面

登录页和各角色主页使用统一视觉风格，突出书店与数据管理场景。

快速定位

主页按钮按高频业务排列，减少进入采购、库存、销售和统计页面的路径成本。

多角色工作台

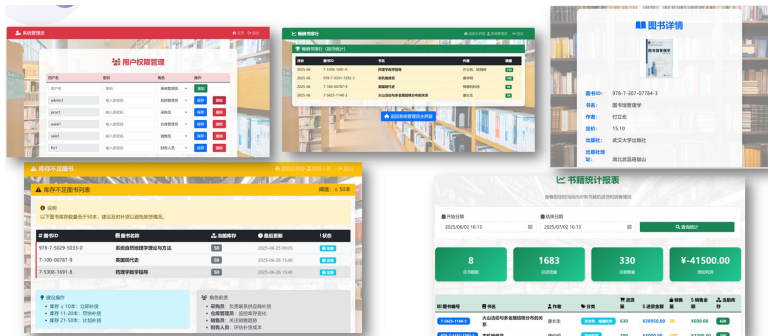


统一入口：各角色主页保持相同结构。

高频操作：订单、库存、预警、统计入口直接呈现。

角色边界：登录后进入对应业务空间。

业务页面与运行验证



运行验证: /login 返回
200, 服务监听
127.0.0.1:5000。

数据库验证: GaussDB
select 1 成功, users 表可
读。

登录验证: 不同角色账号
可进入对应主页。

问题不足与优化方向

工程优化

- ▶ 配置安全: `secret_key` 和数据库配置迁移到环境变量
- ▶ 权限复用: 抽取 `role_required` 装饰器
- ▶ 连接效率: 引入连接池或应用级连接管理

安全与维护

- ▶ 表单安全: 增加 CSRF 防护和输入校验
- ▶ 模块拆分: 按角色拆分 Flask Blueprint
- ▶ 展示优化: 补充关键业务流程的现场演示

谢谢各位

感谢观看

敬请老师批评指正